DOCUMENTO DE DISEÑO · ENTREGABLE 1

Sistema de Memoria Compartida Distribuida (DSM)

Paginación bajo demanda y coherencia MSI

Sebastián Agüero Brenes

José Daniel Monterrosa Quirós

Noviembre de 2026

Abstract

This document presents the design of a distributed shared memory (DSM) system for Linux, implemented in user space and written in C11. The system allows several applications running on independent nodes connected through TCP/IP to share a logical address space even though no physical memory is common to them. Each node contributes a pool of 256 frames of 4 KB (1 MB) and reserves a 256 MB virtual window in which it materializes, on demand, the pages it uses. Accesses to absent or unprotected pages are detected with mmap and mprotect and are caught by a SIGSEGV handler installed through sigaction with SA_SIGINFO; the handler only enqueues the fault, while a resolver thread runs the network protocol and adjusts page permissions. Coherence is maintained with the MSI protocol (Modified, Shared, Invalid) with write invalidation over a distributed directory, in which each page has a home node that serializes its transitions. Concurrency is handled with Pthreads, mutexes, POSIX semaphores and a global lock hierarchy that prevents deadlocks. The document describes the architecture, the memory model, the message protocol and its 64-byte header, the API, the data structures and their sizes, the configuration, the test applications —matrix multiplication, video scene-cut detection and concurrent increments—, the test plan, the work plan and the risks. This is a design document: the code and the experimental results belong to deliverables 2 and 3 and to the final demo.

Índice

Índice 2
1. Introducción 4
2. Antecedentes 4
3. Decisiones principales de diseño 4
4. Arquitectura general 5
4.1 Componentes de un nodo 5
4.2 Ciclo de vida de un nodo 6
5. Espacio de direcciones global 7
5.1 Memoria aportada y caché de páginas ajenas 7
6. Asignación de memoria y segmentos 7
7. Modelo de coherencia MSI 8
7.1 El directorio en el nodo hogar 8
7.2 Invariantes 9
8. Manejo de los fallos de página 9
8.1 Fallos que se resuelven sin salir del nodo 10
8.2 Escritura con invalidaciones 10
9. Protocolo de mensajes entre nodos 11
9.1 Catálogo de mensajes 11
9.2 Cabecera y framing 11
9.3 Transporte 12
10. Membresía y detección de caídas 12
11. Concurrencia, sincronización e interbloqueos 14
11.1 Productor y consumidor con semáforos POSIX 14
11.2 La tabla de páginas 14
11.3 Señales y sockets 14
11.4 Prevención de interbloqueos 15
12. API del DSM 15
13. Estructuras y tipos de datos 16
14. Archivo de configuración 17
15. Consultas y monitoreo 17
16. Aplicaciones de prueba 18
16.1 Detección de cortes de escena en un video 18

16.2 Multiplicación de matrices 18
16.3 Plan de pruebas y métricas 18
17. Diseño previsto para los extras 18
17.1 Ejecución distribuida de procesos 19
17.2 Tolerancia a la caída de un nodo 19
18. Organización del código y del trabajo 19
19. Plan de trabajo 19
20. Riesgos y cómo se mitigan 20
Referencias 20

1. Introducción

Este documento presenta el diseño del sistema de **Memoria Compartida Distribuida (DSM)** del proyecto de IC-6600 Principios de Sistemas Operativos. El sistema emula, sobre varios nodos Linux conectados por TCP/IP, un espacio de memoria compartida: una aplicación lee y escribe punteros normales y el runtime se encarga de traer las páginas, moverlas por la red y mantenerlas coherentes.

El objetivo del Entregable 1 es dejar el diseño suficientemente detallado para implementar los prototipos de los Entregables 2 y 3: arquitectura, modelo de memoria, protocolo MSI, mensajes, concurrencia, API, estructuras de datos, configuración, aplicaciones de prueba, plan de trabajo y riesgos. Es un diseño técnico; no incluye código completo ni resultados de ejecución.

El sistema se escribe en C11, usa `mmap`/`mprotect` para controlar los permisos de las páginas, `sigaction` con `SA_SIGINFO` para capturar los fallos de página, sockets TCP para la comunicación, y Pthreads con semáforos POSIX para la concurrencia.

2. Antecedentes

La idea de una memoria virtual compartida entre máquinas sin memoria física común fue introducida por Li y Hudak (1989) con el sistema IVY, que usa páginas como unidad de coherencia y un protocolo de invalidación de escritura. TreadMarks (Amza et al., 1996) y Munin (Carter et al., 1991) exploraron modelos de consistencia relajados para reducir el tráfico y el *false sharing*. El enfoque con directorio para la coherencia proviene del trabajo sobre cachés (Agarwal et al., 1988).

Este proyecto adopta la versión más simple y verificable de esas ideas:

- **Granularidad de página** (4 KB por defecto), alineada con la MMU del sistema operativo: el hardware detecta el acceso y el runtime reacciona mediante `SIGSEGV`.
- **Protocolo MSI** (*Modified, Shared, Invalid*) con invalidación de escritura.
- **Directorio distribuido**: cada página tiene un nodo hogar (*home*) que registra quién la posee y quién tiene copias.
- **Consistencia secuencial por página**, apoyada en la sincronización explícita (barreras, mutex, operaciones atómicas) para el resto.

3. Decisiones principales de diseño

Tabla 1. Decisiones principales de diseño

Decisión	Alternativas	Motivo
Lenguaje C11	C99, C17	Confirmado por el profesor.
Runtime embebido en el proceso de la aplicación	Daemon separado	El `SIGSEGV` ocurre en el proceso que accede; un proceso aparte no puede resolverlo sin cooperación.
Ventana local + pool de marcos	Misma dirección virtual en todos los nodos	No exige direcciones idénticas; el protocolo usa `page_id`, no punteros.
MSI simplificado	MESI, MOESI	Menos estados; suficiente para el alcance del curso.
Directorio distribuido (home por página)	Directorio central	Evita cuello de botella y punto único de fallo.
TCP con conexiones persistentes	UDP, conexión por mensaje	Entrega confiable y ordenada, menos overhead.

Decisión	Alternativas	Motivo
Página de 4 KB, 1 MB por nodo	8 KB o 64 KB; 4 MB o 16 MB	Menos *false sharing*; 256 marcos por nodo demuestran la paginación en el laboratorio.
Reemplazo LRU aproximado	Clock, aleatorio	Simplicidad y buena localidad.
4 nodos ideales (mínimo 2)	Solo 2	Equilibrio entre demostración y recursos disponibles.
CLI y panel web de solo lectura	Solo CLI	Confirmado por el equipo.
Extras condicionados al tiempo	Extras obligatorios	La base primero; los extras después del Entregable 3.

4. Arquitectura general

El sistema no tiene un servidor central permanente. Cada nodo es un proceso Linux que contiene el runtime DSM enlazado con la aplicación. Un nodo semilla se usa solo para el arranque (asignar `node_id` y `object_id`). Los nodos se comunican por TCP formando una malla; pueden ejecutarse varios nodos en una misma máquina (puertos distintos) o en máquinas distintas.

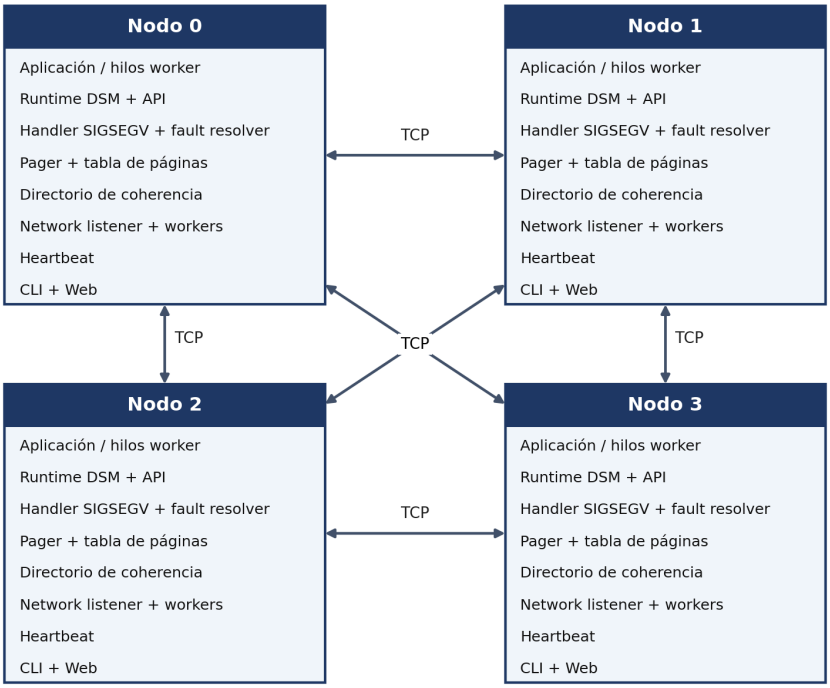


Figura 1. Vista de despliegue: nodos del clúster y comunicación TCP.

4.1 Componentes de un nodo

Tabla 2. Componentes de un nodo

Componente	Responsabilidad
Aplicación y API DSM	Crea y adjunta objetos, reserva memoria, sincroniza; accede con punteros locales.

Componente	Responsabilidad
Handler de `SIGSEGV`	Captura el fallo, calcula la página y encola la solicitud. Hace lo mínimo (async-signal-safe).
`fault_resolver`	Resuelve fallos: consulta la tabla, pide la página por red, instala datos y ajusta permisos.
Tabla de páginas local	Estado MSI, residencia, versión y marco de cada página conocida por el nodo.
Directorio de coherencia	Para las páginas cuyo `home` es este nodo: propietario, copias compartidas, versión.
Pool de marcos	Marcos de memoria física aportados (1 MB por defecto = 256 marcos de 4 KB).
`network_listener` y `network_workers`	Aceptan conexiones y atienden mensajes del protocolo.
`heartbeat`	Envía latidos y detecta nodos caídos.
`stats`, CLI y web	Estadísticas, consultas de solo lectura y panel de monitoreo.

4.2 Ciclo de vida de un nodo

El nodo pasa por `INIT`, carga y valida la configuración (`CONFIG_LOADED`), contacta a la semilla (`JOINING`), recibe su identificador y la lista de nodos (`JOINED`) y arranca sus hilos (`RUNNING`). Si un par es sospechoso o hay fallos de red, pasa a `DEGRADED` y puede volver a `RUNNING` o terminar en `OFFLINE`. El cierre ordenado (`SHUTDOWN`) escribe de vuelta las páginas modificadas y cierra las conexiones.

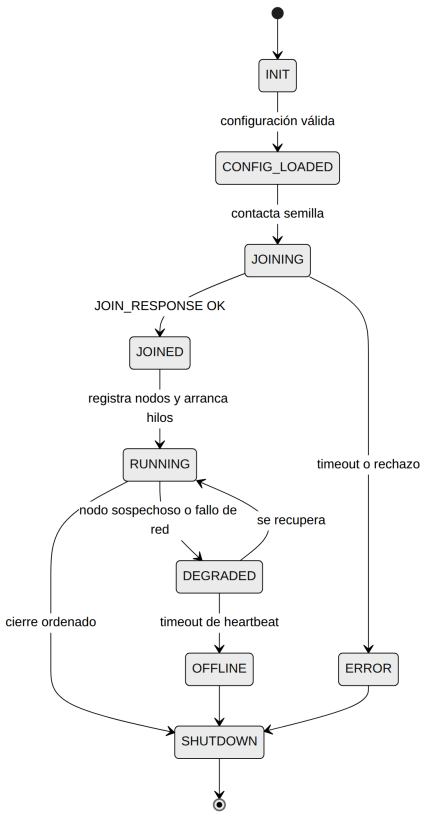


Figura 2. Ciclo de vida de un nodo.

5. Espacio de direcciones global

Existen dos tipos de direcciones. La **dirección local** es una dirección virtual dentro de la ventana del nodo y solo es válida en ese proceso. El **identificador global de página** (`page_id`) es válido en todo el sistema y es lo único que viaja por la red:

```
dirección_local = objeto->local_base + page_index * page_size
page_id         = objeto->first_page_id + page_index
```

Cada nodo reserva una **ventana** de direcciones virtuales (256 MB por defecto) con `mmap(PROT_NONE, MAP_ANONYMOUS | MAP_NORESERVE)`. La ventana es virtual y no consume memoria física hasta que se materializan páginas. Como el protocolo usa `page_id`, el mismo objeto puede estar en direcciones locales distintas en cada nodo.

5.1 Memoria aportada y caché de páginas ajenas

Cada nodo aporta un **pool de marcos** de tamaño `node_memory`. El pool es la memoria física del DSM:

```
frames_per_node = node_memory / page_size      (1 MB / 4 KB = 256)
total_frames    = suma de frames_per_node de todos los nodos
```

Un nodo puede mantener residentes, a lo sumo, `frames_per_node` páginas a la vez, sean propias o de otros nodos. Las páginas ajenas que un nodo trae por demanda funcionan como una **caché**: llegan en estado `S` (lectura) o `M` (escritura) y se expulsan con LRU aproximado cuando se necesita un marco. Una página `M` se escribe de vuelta al home antes de liberar el marco (`madvise(MADV_DONTNEED)`); si no hay marcos libres ni páginas expulsables, la operación falla con `DSM_ERR_NOMEM`.

6. Asignación de memoria y segmentos

La unidad de reserva es el **objeto compartido** (el segmento): tiene `object_id`, nombre lógico, tamaño, rango de `page_id`, nodo creador y una base local por nodo.

- `dsm_create` reserva el rango de `page_id` de forma contigua y asigna el `home_node` de cada página en round-robin entre los nodos con marcos disponibles (o según la política configurada).
- `dsm_attach` permite que otro nodo se adjunte por nombre; mapea el objeto en su ventana sin traer datos.
- Las páginas se materializan **bajo demanda**: la primera vez que un hilo toca una página se produce un fallo y se resuelve con el protocolo.
- Dentro de un objeto, `dsm_malloc` usa un *bump allocator* alineado a 16 bytes (`dsm_free` es un no-op en la versión base; se documenta como limitación). Alcanza para matrices, archivos y contadores.

> **Advertencia.** `dsm_malloc` devuelve una dirección **local**. Para compartir referencias entre nodos se usan *offsets* dentro del objeto (`dsm_offset` / `dsm_ptr`), nunca punteros.

7. Modelo de coherencia MSI

La unidad de coherencia es la página. Cada nodo mantiene, para cada página, uno de tres estados que determinan los permisos locales:

Tabla 3. Estados MSI y permisos locales

Estado	Significado	Permisos (`mprotect`)
M (*Modified*)	Copia exclusiva y más reciente.	`PROT_READ | PROT_WRITE`
S (*Shared*)	Copia de solo lectura, posiblemente en varios nodos.	`PROT_READ`
I (*Invalid*)	Sin copia válida.	`PROT_NONE`

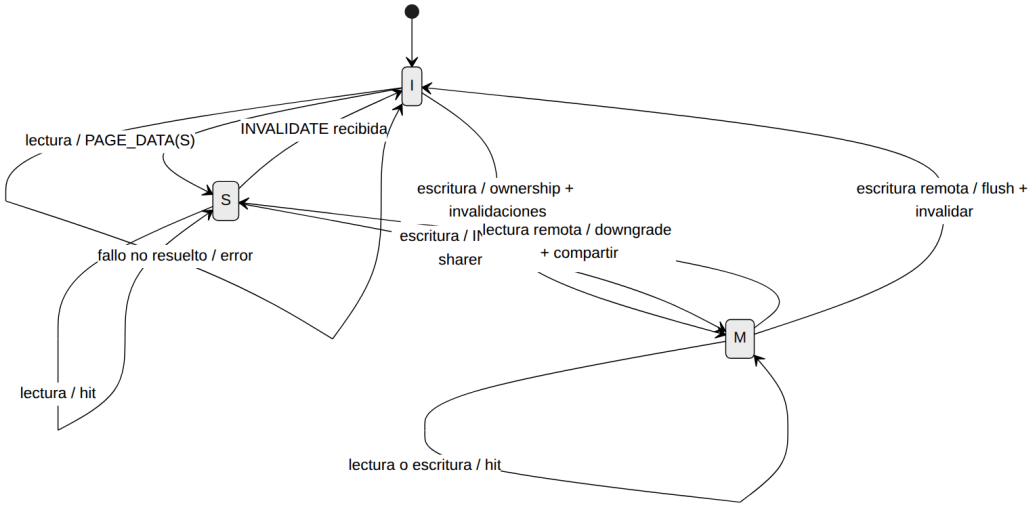


Figura 3. Diagrama de estados MSI de una página en un nodo.

Las transiciones principales son:

Tabla 4. Transiciones principales MSI

Evento	Estado	Acción	Final
Lectura, página ausente	`I`	`PAGE_REQUEST` al home; recibir `PAGE_DATA`	`S`
Escritura, página ausente	`I`	`OWNERSHIP_REQUEST`; invalidar copias; recibir datos	`M`
Escritura, página compartida	`S`	Invalidar demás copias; esperar ACK	`M`
Lectura remota de página propia	`M`	Degradar; enviar datos; registrar sharer	`S`
Escritura remota de página propia	`M`	Enviar datos y propiedad	`I`
Invalidación recibida	`S` o `M`	Descartar (escribir de vuelta si `M`); `INVALIDATE_ACK`	`I`
Expulsión	`S` o `M`	Descartar o *write-back* al home	`I`

7.1 El directorio en el nodo hogar

Cada página tiene un `home_node`. El home mantiene una entrada de directorio (`dsm_directory_entry_t`, 64 bytes) con el estado, el `owner_node`, el bitmap de `sharers` (64 bits, hasta 64 nodos), la versión y la solicitud pendiente. El home **no necesariamente tiene los datos**: si el propietario es otro nodo, redirige la solicitud

(`PAGE_FORWARD`). Todas las transiciones de una página se serializan en su home, lo que evita carreras entre solicitudes concurrentes.

7.2 Invariantes

1. Para cada página existe como máximo un nodo en `M`.
2. Si un nodo está en `M`, ningún otro nodo tiene una copia válida.
3. Si hay varios nodos en `S`, las copias son idénticas y no hay ningún `M`.
4. Antes de pasar de `S` a `M` (o de conceder `M`) se invalidan todas las demás copias y se reciben sus ACK.
5. Una página `M` se escribe de vuelta antes de perderse por expulsión, cierre o fallo.
6. La versión de una página solo aumenta cuando se modifica su contenido.

8. Manejo de los fallos de página

El hilo de aplicación accede a una dirección de la ventana sin permiso suficiente y el kernel entrega `SIGSEGV`. El handler (instalado con `sigaction` y `SA_SIGINFO`) lee `siginfo_t.si_addr`, calcula el `page_id`, encola el fallo, avisa al `fault_resolver` con `sem_post` y suspende al hilo. El resolver obtiene la página, instala los datos, ajusta permisos con `mprotect` y despierta al hilo, que reintenta la instrucción.

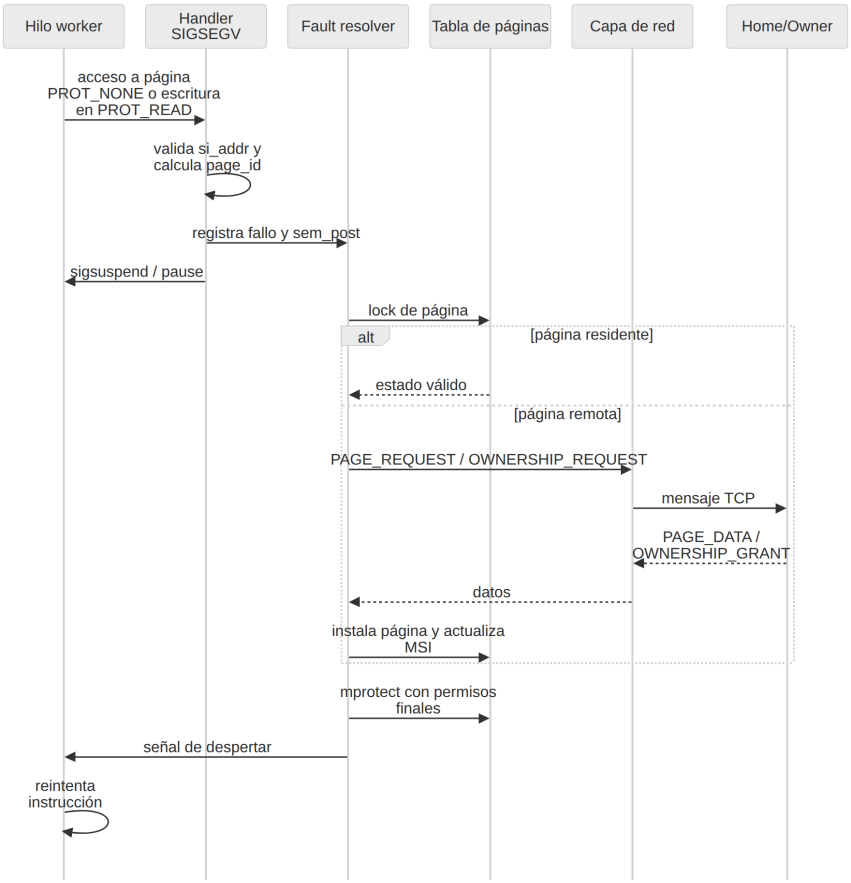


Figura 4. Flujo de resolución de un fallo de página.

El handler debe ser mínimo: solo valida la dirección, encola y hace `sem_post` (async-signal-safe). Nunca toma mutex ni hace red. Si `si_addr` no pertenece a la ventana DSM, el fallo se reenvía al comportamiento por

defecto para no ocultar errores reales. Para distinguir lectura de escritura se examina el registro de error del `ucontext`, según la arquitectura.

8.1 Fallos que se resuelven sin salir del nodo

No todo fallo requiere red:

- **Página ya residente con permisos insuficientes**: por ejemplo, escritura sobre una página `S` cuyo único sharer es este nodo; solo cambia el estado.
- **Página en vuelo**: si otro hilo ya pidió la misma página, el fallo se agrupa (*coalescing*) y el hilo espera la misma respuesta en la variable de condición de la página.
- **Página local con datos**: si este nodo es el home y tiene la copia, se atiende desde la tabla local.
- **Primera materialización de una página nueva**: se asigna un marco puesto en cero sin tráfico de red.

8.2 Escritura con invalidaciones

Cuando un nodo necesita escribir una página compartida, pide la propiedad al home. El home bloquea las transiciones de esa página, envía `INVALIDATE` a cada sharer, espera todos los `INVALIDATE_ACK`, actualiza el directorio (propietario = solicitante, sharers = ∅) y concede `OWNERSHIP_GRANT` con los datos. Solo entonces el solicitante pasa a `M`.

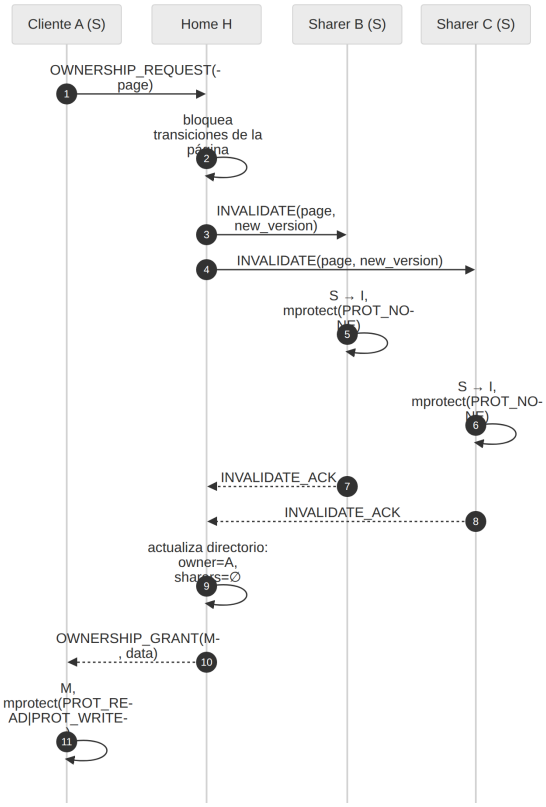


Figura 5. Escritura con invalidación de todas las copias.

9. Protocolo de mensajes entre nodos

Todos los mensajes comparten una cabecera fija de 64 bytes seguida de un payload de longitud variable. Los mensajes viajan en orden por conexiones TCP persistentes entre cada par de nodos.

9.1 Catálogo de mensajes

Tabla 5. Catálogo de mensajes

Código	Mensaje	Dirección	Función
`0x0001`	`HELLO` / `HELLO_ACK`	Nodo ↔ Nodo	Presentación y aceptación.
`0x0003`	`JOIN_REQUEST` / `JOIN_RESPONSE`	Nodo ↔ Semilla	Ingreso al clúster; asigna `node_id`.
`0x0005`	`NODE_JOIN_BROADCAST`	Nodo → Todos	Anuncia un nodo nuevo.
`0x0006`	`NODE_LEAVE`	Nodo → Todos	Salida ordenada.
`0x0010`	`HEARTBEAT` / `HEARTBEAT_ACK`	Nodo ↔ Nodo	Latido y estado.
`0x0020`	`PAGE_REQUEST`	Nodo → Home	Solicita una página para lectura.
`0x0021`	`PAGE_DATA`	Home/Owner → Nodo	Entrega la página.
`0x0022`	`PAGE_FORWARD`	Home → Owner	Redirige una solicitud al propietario.
`0x0023`	`PAGE_WRITEBACK` / `_ACK`	Nodo ↔ Home	Devuelve una página `M` antes de expulsarla.
`0x0030`	`OWNERSHIP_REQUEST`	Nodo → Home	Solicita escribir (propiedad).
`0x0031`	`OWNERSHIP_GRANT`	Home → Nodo	Concede propiedad y datos.
`0x0032`	`OWNERSHIP_DENY`	Home → Nodo	Rechaza la solicitud.
`0x0040`	`INVALIDATE` / `INVALIDATE_ACK`	Home ↔ Sharer	Invalida una copia y confirma.
`0x0050`	`OBJECT_CREATE` / `ATTACH` / `RELEASE`	Nodo ↔ Semilla/Home	Gestión de objetos y sus `_ACK`.
`0x0060`	`STATS_REQUEST` / `_RESPONSE`	Nodo ↔ Nodo	Estadísticas.
`0x0070`	`NODE_STATUS_REQUEST` / `_RESPONSE`	Nodo ↔ Nodo	Estado de un nodo.
`0x0080`	`ERROR` / `ACK`	Cualquiera	Error de protocolo / confirmación genérica.
`0x0090`	`SHUTDOWN` / `_ACK`	Admin ↔ Nodo	Cierre ordenado.

9.2 Cabecera y framing

El diseño usa un **runtime embebido**, por lo que no existe un canal local entre una biblioteca y un daemon: la aplicación llama directamente al API DSM dentro de su propio proceso, y los hilos internos se coordinan con estructuras compartidas y semáforos. El único canal entre procesos es TCP; la CLI se comunica con el nodo por su puerto de control.

Cada mensaje inicia con `dsm_msg_header_t` (64 bytes):

Tabla 6. Campos de la cabecera de mensaje

Campo	Tamaño	Descripción
`request_id`	8	Identifica solicitud y respuesta; permite reintentos idempotentes.

Campo	Tamaño	Descripción
`page_id`	8	Página a la que se refiere (0 si no aplica).
`magic`, `version`	4 + 2	Validación del marco y versión del protocolo.
`type`, `flags`	2 + 4	Código del mensaje y banderas (respuesta, reintento, con datos...).
`header_len`, `payload_len`	4 + 4	Longitudes; el marco total es `header_len + payload_len`.
`src_node`, `dst_node`	4 + 4	Origen y destino.
`status`, `checksum`	4 + 4	Resultado (respuestas) y CRC32 del payload.
`payload_off`, `ttl`, reservados	4 + 2 + 2 + 4	Offset de datos, saltos restantes y relleno.

El *framing* es por longitud: el receptor lee primero la cabecera (64 bytes) con `recv_all`, valida `magic` y `version`, y luego lee exactamente `payload_len` bytes. Los campos viajan en orden de bytes de red (*network byte order*).

9.3 Transporte

- **TCP con conexiones persistentes** por par de nodos, abiertas la primera vez que se necesitan y reutilizadas.
- Cada nodo reserva un rango de cuatro puertos consecutivos (datos, control, heartbeat y web) dentro de `port_range_start..port_range_end`.
- `send_all` y `recv_all` reintentan ante `EINTR` y envíos parciales; `MSG_NOSIGNAL` evita `SIGPIPE`.
- Cada solicitud tiene un timeout (`network_timeout_ms`) y un número de reintentos (`retry_count`). Como las solicitudes llevan `request_id`, el receptor descarta duplicados y las respuestas son idempotentes.
- Los datos de página se verifican con CRC32; un error de checksum produce `ERROR` y reintento.

10. Membresía y detección de caídas

Ingreso. El nodo nuevo envía `JOIN_REQUEST` a la semilla, que valida versión y parámetros (`page_size`, límite de nodos), le asigna `node_id` y devuelve la lista de nodos. El nodo nuevo anuncia su llegada con `NODE_JOIN_BROADCAST` y comienza a enviar heartbeats.

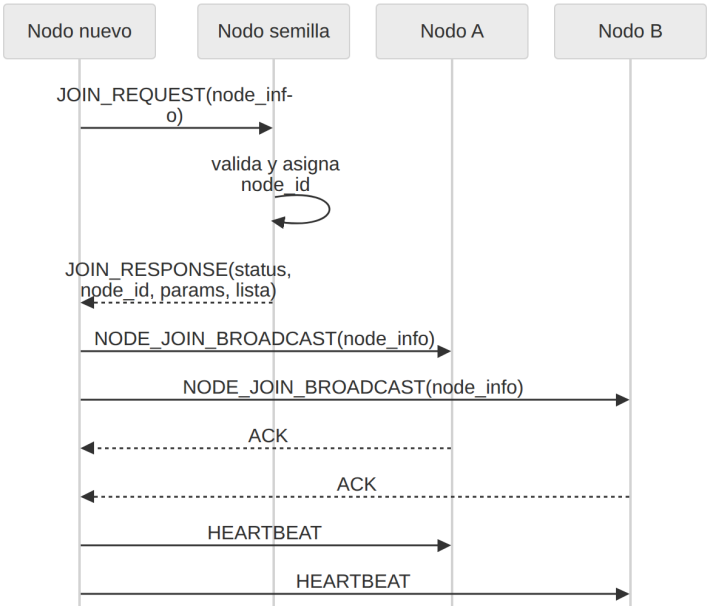


Figura 6. Incorporación de un nodo al clúster.

Tabla de nodos. Cada nodo guarda, por par, su `node_id`, dirección, memoria aportada, estado (`ONLINE`, `SUSPECT`, `OFFLINE`), último latido, carga y versión del protocolo.

Detección. Cada `heartbeat_interval_ms` se envía un `HEARTBEAT`. Si no hay respuesta dentro del timeout, el nodo pasa a `SUSPECT`; si falla un segundo intento, pasa a `OFFLINE` y se invalidan las páginas cuyo propietario era ese nodo. Un nodo `OFFLINE` no recibe nuevas solicitudes y sus páginas `M` quedan no disponibles, salvo que exista réplica (extra 2).

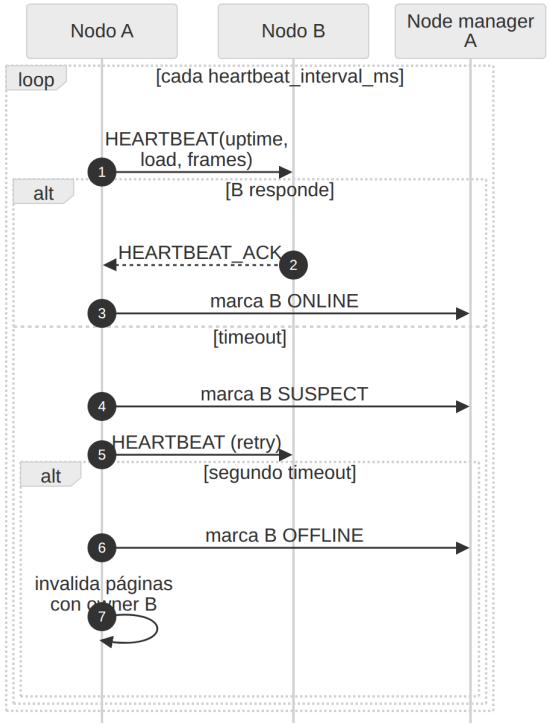


Figura 7. Heartbeat y detección de un nodo caído.

11. Concurrencia, sincronización e interbloqueos

Un nodo ejecuta varios tipos de hilos: los hilos de la aplicación (*workers*), el `network_listener`, los `network_workers`, el `fault_resolver`, el `heartbeat`, `stats` y `web`. Las estructuras compartidas entre ellos se protegen con la primitiva adecuada: `pthread_mutex_t` para exclusión corta, `pthread_rwlock_t` para la tabla de nodos y la configuración (muchas lecturas), `pthread_cond_t` para esperar páginas en vuelo, `sem_t` sin nombre para colas y marcos libres, y contadores atómicos para estadísticas.

11.1 Productor y consumidor con semáforos POSIX

El patrón central es **productor-consumidor** entre el handler de `SIGSEGV` y el `fault_resolver`:

- El **productor** es el handler: escribe un `fault_req` en una cola circular acotada y llama a `sem_post(&fault_sem)`. `sem_post` es *async-signal-safe*, por eso es la única operación de sincronización permitida dentro del handler.
- El **consumidor** es el `fault_resolver`: espera con `sem_wait(&fault_sem)`, extrae la solicitud y la resuelve.
- Un segundo semáforo cuenta los **marcos libres** del pool: quien necesita un marco hace `sem_wait`; la expulsión y la liberación hacen `sem_post`. Si el semáforo bloquea demasiado, se intenta expulsar otra página.
- La cola es acotada, de modo que un nodo saturado no acumula solicitudes sin límite.

11.2 La tabla de páginas

Cada entrada de la tabla tiene su propio mutex y variable de condición (granularidad fina), más un lock por bucket de la tabla hash. Las transiciones siguen siempre el mismo orden: se toma el lock de la página, se verifica residencia y estado, se marca `FETCHING` y se **suelta el lock**; la solicitud de red se hace sin locks; al llegar la respuesta se vuelve a tomar el lock, se instalan los datos, se cambia el estado MSI y se hace `broadcast` de la condición.

> **Regla.** Nunca se mantiene `page->lock` durante una llamada de red: un nodo lento bloquearía a todos los hilos que tocan esa página y podría producir un interbloqueo distribuido.

11.3 Señales y sockets

La convivencia entre `SIGSEGV` y la red es el punto más delicado:

Tabla 7. Problemas entre señales y sockets

Problema	Solución
El handler toma un mutex que tenía el hilo interrumpido (interbloqueo).	El handler solo encola y hace `sem_post`.
`SIGSEGV` interrumpe `accept`, `recv` o `send`.	`SA_RESTART` y reintento explícito; `recv_all`/`send_all`.
`SIGPIPE` al escribir en un socket cerrado.	`MSG_NOSIGNAL` o `SIGPIPE` ignorada.
La señal llega a un hilo interno.	`pthread_sigmask`: los hilos internos bloquean `SIGSEGV`.
Reentrada en el handler o fallo real (bug).	Bandera `in_fault` por hilo y validación de `si_addr`.

11.4 Prevención de interbloqueos

Se define un **orden global de adquisición de locks**; un hilo solo puede tomar locks de nivel mayor al que ya tiene:

Tabla 8. Orden global de adquisición de locks

Nivel	Lock
1	`g_shutdown_lock`
2	`g_node_table_lock`
3	`g_directory_lock` (bucket)
4	`page->lock`
5	`g_frame_lock`
6	`conn->tx_lock`
7	`g_stats_lock`
8	`g_log_lock`

Además: no se hace red con locks tomados; los locks de página y de conexión usan `pthread_mutex_timedlock` (500 ms y 1000 ms) con reintento; los caminos de cierre usan `trylock`; las colas son acotadas; y cada transición de página tiene un único propietario serializado por el home. Opcionalmente, un modo de diagnóstico construye un grafo de espera para detectar ciclos.

12. API del DSM

El API devuelve punteros normales y códigos de estado (`DSM_OK` o negativos como `DSM_ERR_INVAL`, `DSM_ERR_NOMEM`, `DSM_ERR_NET`, `DSM_ERR_TIMEOUT`).

Tabla 9. Funciones del API DSM

Grupo	Funciones	Descripción
Inicio y cierre	`dsm_init`, `dsm_shutdown`, `dsm_node_id`	Reserva la ventana, instala el handler, arranca hilos y se une al clúster.
Objetos	`dsm_create`, `dsm_attach`, `dsm_detach`, `dsm_destroy`	Crean, adjuntan y liberan objetos compartidos por nombre.
Memoria	`dsm_malloc`, `dsm_calloc`, `dsm_free`, `dsm_ptr`, `dsm_offset`	Reserva dentro de un objeto y traduce entre offset y puntero local.
Sincronización	`dsm_barrier_create/wait/destroy`, `dsm_mutex_create/lock/unlock/destroy`, `dsm_atomic_add`	Barreras, mutex distribuidos y suma atómica sobre una página.
Consulta	`dsm_stats`, `dsm_nodes`, `dsm_pages`, `dsm_objects`, `dsm_node_status`	Lectura de estadísticas y tablas; seguras desde CLI y web.
Configuración	`dsm_config_defaults/load/validate/save`, `dsm_strerror`, `dsm_set_log_level`	Manejo de configuración, mensajes de error y nivel de log.

Firmas principales:

```
dsm_status_t dsm_init(const dsm_config_t *cfg);
dsm_status_t dsm_create(const char *name, size_t size,
                        uint32_t flags, dsm_object_t **out);
void        *dsm_malloc(dsm_object_t *obj, size_t size);
dsm_status_t dsm_barrier_wait(dsm_barrier_t *b, int timeout_ms);
dsm_status_t dsm_stats(dsm_stats_t *out);
```

`dsm_init` debe llamarse una vez, antes de crear hilos de usuario. Las funciones de consulta son de solo lectura.

13. Estructuras y tipos de datos

Las estructuras que cruzan la red o se comparten entre módulos usan tipos de ancho fijo, tamaños múltiplos de 8 y `_Static_assert` sobre `sizeof` para detectar diferencias de ABI. Los tamaños se verificaron compilando las definiciones.

Tabla 10. Tamaño de las estructuras principales

Estructura	Bytes	Uso
`dsm_msg_header_t`	64	Cabecera de todo mensaje.
`dsm_node_info_t`	112	Tabla de nodos.
`dsm_page_info_t`	64	Tabla de páginas local.
`dsm_directory_entry_t`	64	Directorio del home.
`dsm_object_t`	112	Objeto compartido.
`dsm_frame_t`	40	Marco del pool.
`dsm_stats_t`	128	Estadísticas por nodo.
`dsm_config_t`	704	Configuración en memoria.

Por ejemplo, la entrada de la tabla de páginas es:

```
typedef struct dsm_page_info {
    uint64_t page_id;          /*  0 */
    uint64_t object_id;        /*  8 */
    uint64_t version;          /* 16 */
    uint64_t sharers;          /* 24: bitmap de nodos */
    uint64_t last_access_ms;   /* 32 */
    uint32_t home_node;        /* 40 */
    uint32_t owner_node;       /* 44 */
    uint32_t refcount;         /* 48 */
    uint32_t frame_slot;       /* 52 */
    uint8_t  state;            /* 56: M / S / I */
    uint8_t  is_resident;      /* 57 */
    uint8_t  is_dirty;         /* 58 */
    uint8_t  flags;            /* 59 */
    uint32_t reserved;         /* 60 */
} dsm_page_info_t;              /* 64 bytes */
```

La memoria de metadatos por nodo es pequeña: con 256 marcos y 256 páginas locales, tabla, directorio y marcos suman unos 66 KB (alrededor del 6 % de 1 MB). Todas las estructuras definen explícitamente sus offsets y su relleno de alineación.

14. Archivo de configuración

La configuración es un archivo `clave = valor` (comentarios con `#`). Se carga con `dsm_config_load`, se valida y puede sobrescribirse por línea de comandos (precedencia: valores por defecto, archivo, línea de comandos). El modo `--check-config` solo valida y termina.

Tabla 11. Parámetros de configuración

Parámetro	Defecto	Descripción
`node_id`	—	Identificador del nodo (0 = semilla).
`bind_address`	`0.0.0.0`	Dirección de escucha.
`listen_port`, `control_port`, `heartbeat_port`, `web_port`	50000 + 4·n	Puertos del nodo (rango 50000-50099).
`seed_host`, `seed_port`	—	Nodo semilla para el ingreso.
`page_size`	4096	Tamaño de página; potencia de dos, igual en todos los nodos.
`node_memory`	1 MB	Memoria aportada (define el número de marcos).
`window_size`	256 MB	Ventana de direcciones virtuales.
`max_nodes`	16	Máximo de nodos (hasta 64 por el bitmap).
`heartbeat_interval_ms`, `network_timeout_ms`, `retry_count`	según archivo	Detección de caídas y reintentos.
`thread_pool_size`, `log_level`, `log_file`	—	Hilos de red y registro.

```
# dsm.conf — nodo 0 (semilla)
node_id = 0
listen_port = 50000
seed_host = 127.0.0.1
seed_port = 50000
page_size = 4096
node_memory = 1048576
window_size = 268435456
```

La validación rechaza valores incoherentes (por ejemplo, `page_size` que no sea potencia de dos, puertos fuera del rango o `window_size` menor que `node_memory`) con un mensaje que indica el parámetro erróneo.

15. Consultas y monitoreo

Cada nodo expone información de solo lectura por tres vías:

- **API**: `dsm_stats`, `dsm_nodes`, `dsm_pages`, `dsm_objects`, `dsm_node_status`.
- **CLI (`dsm-cli`)**: se conecta al puerto de control del nodo. Comandos: `status`, `nodes`, `objects`, `pages`, `page <id>`, `stats` (con `--watch`), `log-level` y `shutdown`; la salida puede ser tabla o JSON.

- **Web**: panel HTML y endpoints JSON de solo lectura (`/api/status`, `/api/nodes`, `/api/objects`, `/api/pages`, `/api/stats`, `/api/metrics`, `/healthz`).

Las estadísticas por nodo (`dsm_stats_t`) incluyen fallos de página, aciertos locales, lecturas y escrituras remotas, invalidaciones enviadas y recibidas, expulsiones, bytes y mensajes transferidos, timeouts, errores, páginas residentes y marcos libres. El panel web no implementa autenticación en la versión base.

16. Aplicaciones de prueba

Se adaptan tres aplicaciones al API DSM. Las dos primeras son intensivas en memoria; la tercera es la prueba de estrés de coherencia.

16.1 Detección de cortes de escena en un video

Un archivo binario (video o imagen grande) se carga en un objeto DSM y se procesa en paralelo por bloques de 64 KB. Cada nodo toma un conjunto de bloques y calcula resultados parciales (checksum o diferencias entre cuadros); al final se combinan y se verifican contra una ejecución secuencial. Los datos de prueba del archivo (ruta, tamaño y checksum esperado) están pendientes de definir.

16.2 Multiplicación de matrices

Se distribuye una matriz de `N × N` valores `double` en el DSM; cada nodo procesa una franja de filas y luego se verifica el resultado. Con `N = 1024` la matriz ocupa 8 MB, que **no cabe** en los 4 MB físicos de cuatro nodos de 1 MB: el DSM debe paginar, expulsar y recargar, lo que demuestra la paginación bajo demanda (modo A). Con una matriz que sí cabe (modo B) se mide el *speedup*. Para evitar *thrashing* extremo, la multiplicación se hace por bloques.

16.3 Plan de pruebas y métricas

Además de las aplicaciones, la prueba de **incrementos concurrentes** (4 nodos × 4 hilos × 10 000 incrementos) debe producir exactamente 160 000, con las variantes `dsm_atomic_add` y `dsm_mutex`.

Tabla 12. Niveles de prueba

Nivel	Qué se prueba
Unitarias	Cálculo de `page_id`, tabla de páginas, directorio, serialización de mensajes, validación de configuración.
Integración	Ingreso de nodos, lectura y escritura remotas, invalidación, expulsión con *write-back*.
Coherencia MSI	Cada transición de la tabla y los invariantes con 2, 3 y 4 nodos.
Estrés	Incrementos concurrentes, matriz bajo carga, expulsión continua.
Fallos	Caída de un nodo, timeouts, mensajes duplicados.

Las métricas son las de `dsm_stats_t` más la latencia de fallo (de `SIGSEGV` a reintento) y de página remota (p50 y p95), con `speedup(n) = T(1) / T(n)` y `eficiencia(n) = speedup(n) / n`.

17. Diseño previsto para los extras

Los extras se atienden solo después de cerrar el Entregable 3 y no alteran la versión base.

17.1 Ejecución distribuida de procesos

Un lanzador permitiría iniciar los hilos o procesos *worker* de una aplicación en nodos distintos, todos adjuntos al mismo objeto DSM. La coordinación usa las barreras y mutex distribuidos existentes; no requiere mensajes nuevos de coherencia.

17.2 Tolerancia a la caída de un nodo

Se prevén réplicas de páginas `M`: el propietario envía `REPLICA_WRITE` a una réplica y periódicamente `CHECKPOINT`, con sus confirmaciones. Los objetos se marcan con `DSM_OBJ_REPLICATED`. Si el propietario cae, el home recupera la página desde la réplica; sin réplica, la página queda no disponible, como en la versión base.

18. Organización del código y del trabajo

El código se organiza en tres módulos que coinciden con las partes del diseño:

Tabla 13. Módulos de implementación

Módulo	Contenido
Módulo 1: memoria y paginación	Ventana, tabla de páginas, `sigaction`, `fault_resolver`, reemplazo.
Módulo 2: red y protocolo	Listener, conexiones, *framing*, mensajes, heartbeat.
Módulo 3: concurrencia	Locks, semáforos, orden de adquisición, cierre ordenado.

La estructura propuesta del repositorio es `include/` (API público), `src/` (runtime), `apps/` (aplicaciones de prueba), `tools/` (CLI), `tests/`, `conf/` y `docs/`, con un `Makefile` en la raíz. El repositorio Git será: https://github.com/TSebas12/ProyectoSO-DSM-SistemaDeMemoriaCompartidaDistribuida

Ambos integrantes deben poder explicar todas las partes; la distribución siguiente es de carga, no de exclusividad.

Tabla 14. Distribución de trabajo

Área	Responsable	Apoyo
Memoria y paginación	José Daniel	Sebastián
Red y protocolo	Sebastián	José Daniel
Coherencia MSI y concurrencia	José Daniel	Sebastián
Aplicaciones, pruebas y métricas	Sebastián	José Daniel
Documentación, demo y defensa	Ambos	Ambos

19. Plan de trabajo

Tabla 15. Calendario

Semana	Fechas	Entregable	Actividades
1	27 oct – 6 nov	**E1**	Diseño, API, protocolo, estructuras, configuración.
2	7 – 13 nov	**E2**	Prototipo 1: memoria, red, MSI básico, 2 nodos.

Semana	Fechas	Entregable	Actividades
3	14 – 20 nov	**E3**	Prototipo 2: concurrencia, estrés, aplicaciones, métricas.
4	21 – 27 nov	**Final**	Regresión, rendimiento, manual, demo y defensa.

Hitos: primera página remota (11 nov), MSI básico con invalidación (13 nov), cuatro nodos estables (18 nov), aplicaciones con resultados correctos (19 nov), pruebas de estrés (20 nov), demo ensayada (25 nov) y demo final (27 nov).

20. Riesgos y cómo se mitigan

Tabla 16. Riesgos y mitigación

Riesgo	Prob.	Impacto	Mitigación
Inestabilidad del handler de `SIGSEGV`.	Alta	Alto	Handler mínimo, `sigaltstack`, pruebas con una sola página.
Interbloqueo entre locks y red.	Alta	Alto	Orden de locks, timeouts, no hacer red con locks.
MSI incompleto.	Media	Alto	Implementar primero `S`, luego `M`, luego invalidación.
Demasiado alcance.	Alta	Medio	Priorizar 2 nodos y una página; extras al final.
Red del laboratorio restringida o pocos equipos.	Media	Alto	Probar puertos; modo multiproceso en una máquina.
Pérdida de datos por expulsión.	Media	Alto	*Write-back* obligatorio antes de liberar el marco.
False sharing y *thrashing*.	Media	Medio	Relleno y partición de datos; ajustar `node_memory` y tamaño del problema.
Demo no reproducible.	Media	Alto	Scripts, configuración de respaldo, video grabado.

Referencias

Agarwal, A., Simoni, R., Hennessy, J., & Horowitz, M. (1988). An evaluation of directory schemes for cache coherence. *Proceedings of the 15th Annual International Symposium on Computer Architecture*, 280–289.

Amza, C., Cox, A. L., Dwarkadas, S., Keleher, P., Lu, H., Rajamony, R., Yu, W., & Zwaenepoel, W. (1996). TreadMarks: Shared memory computing on networks of workstations. *IEEE Computer, 29*(2), 18–28.

Butenhof, D. R. (1997). *Programming with POSIX Threads*. Addison-Wesley.

Carter, J. B., Bennett, J. K., & Zwaenepoel, W. (1991). Implementation and performance of Munin. *Proceedings of the 13th ACM Symposium on Operating Systems Principles*, 152–164.

Li, K., & Hudak, P. (1989). Memory coherence in shared virtual memory systems. *ACM Transactions on Computer Systems, 7*(4), 321–359.

Linux man-pages project. (2024). *mmap(2), mprotect(2), madvise(2), sigaction(2), signal(7), sem_overview(7), pthreads(7), tcp(7)*. https://man7.org/linux/man-pages/

Stevens, W. R., Fenner, B., & Rudoff, A. M. (2004). *UNIX Network Programming, Volume 1: The Sockets Networking API* (3rd ed.). Addison-Wesley.

Tanenbaum, A. S., & van Steen, M. (2017). *Distributed Systems* (3rd ed.). Maarten van Steen.

Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson.